

GitHub Repository AI Assistant

Technical Project Specification, Architecture & Docker-First Implementation Plan

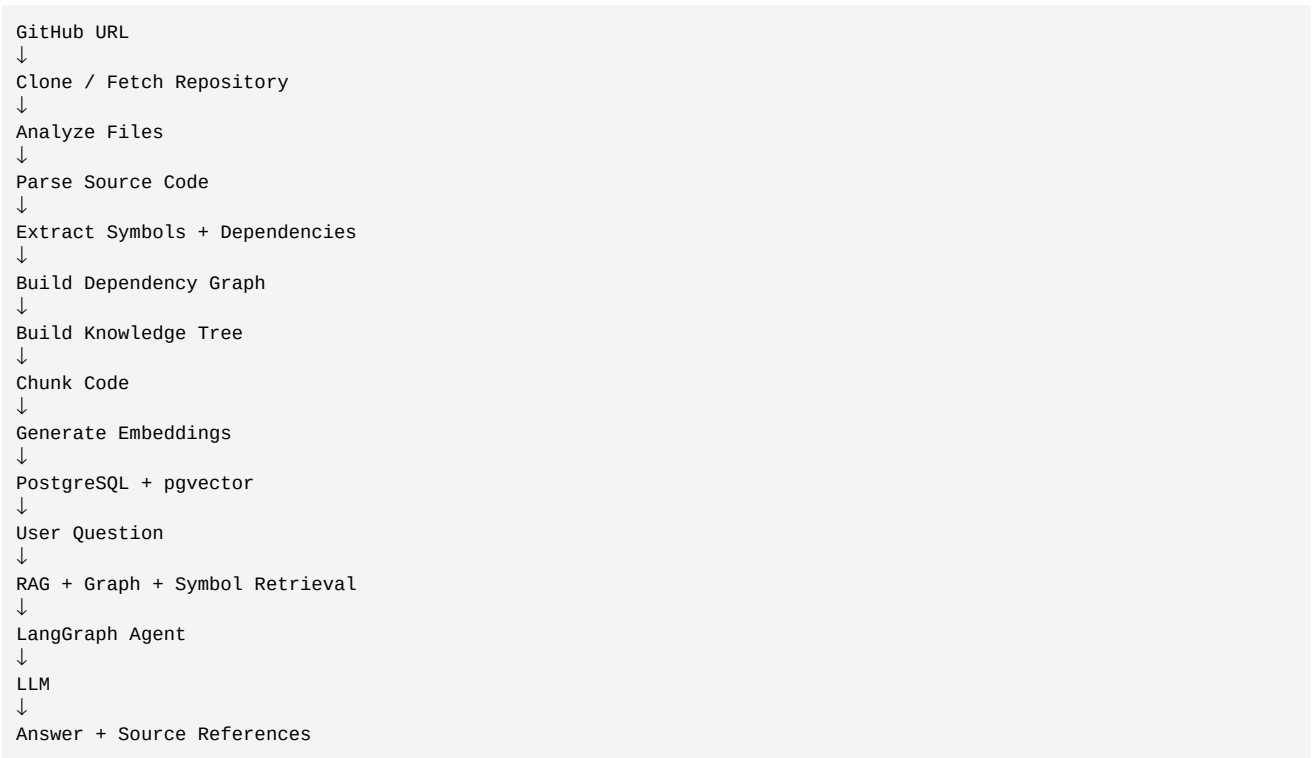
Goal: Accept a public GitHub repository URL, analyze its source code, build a dependency graph and knowledge tree, index the code for RAG, and answer repository questions using AI agents.

Category	Recommended choice
Host	Python + Docker only
Backend	Python + FastAPI
AI orchestration	LangChain + LangGraph
Code analysis	Tree-sitter
Database	PostgreSQL + pgvector
Cache / jobs	Redis
Frontend	React + TypeScript
Graph UI	React Flow
Deployment	Docker Compose

Docker-first rule: Git, Node.js, PostgreSQL, Redis, Java, Maven, LangChain, Tree-sitter and other project dependencies should run inside containers. The host only needs a working Docker Engine / Docker Desktop; Python may be used for auxiliary scripts.

1. Project Overview

This is not just a chatbot. It is a repository-understanding platform with four major capabilities: repository analysis, dependency graph construction, knowledge-tree generation, and RAG/agent-based question answering.



2. Example Questions

- Where is authentication implemented?
- Explain the complete payment or checkout flow.
- Which classes depend on PaymentService?
- Where is PostgreSQL configured?
- What happens when an order is created?
- Where is a specific method defined and where is it called?
- What are the major modules and responsibilities of this repository?
- What could break if a particular service becomes unavailable?

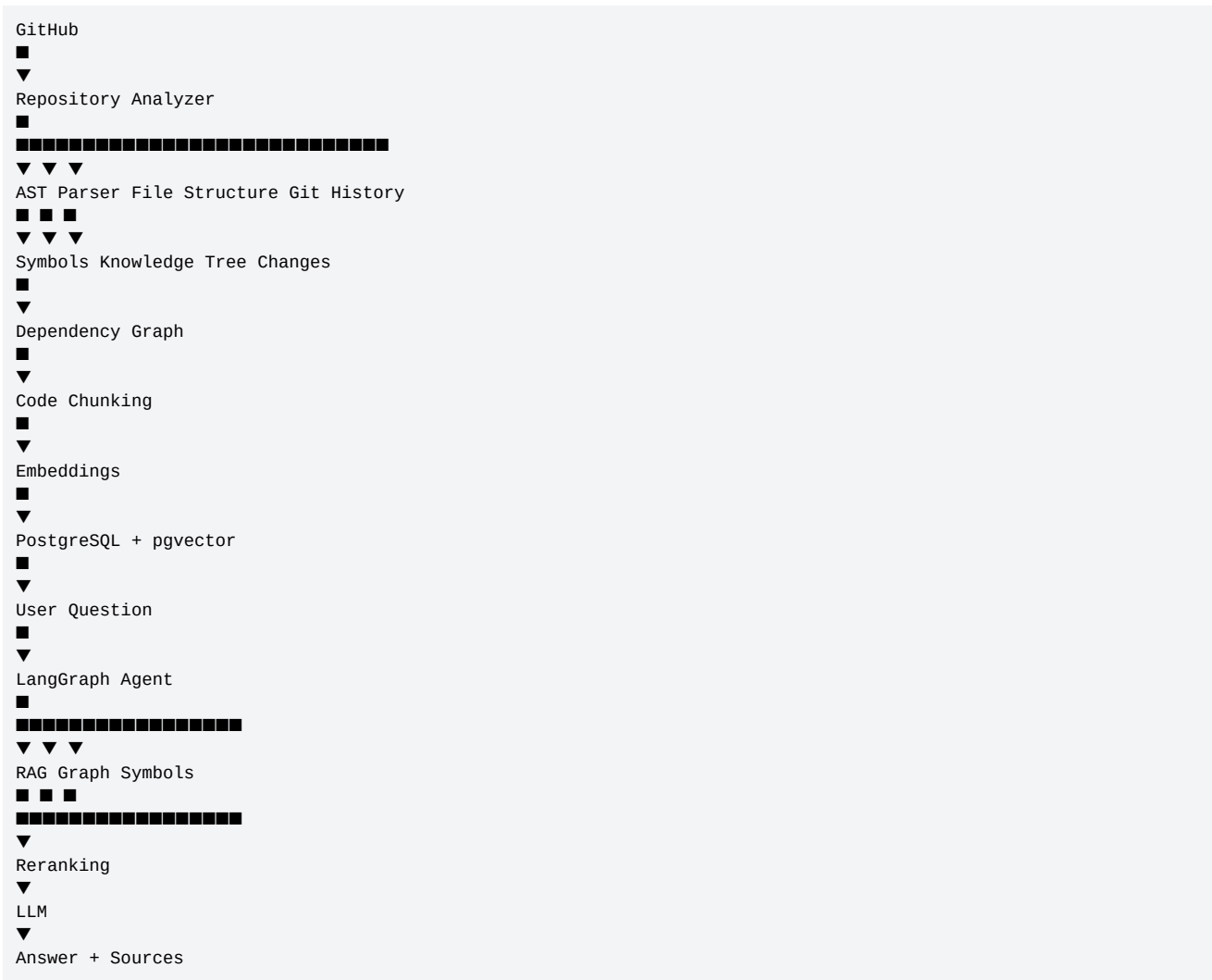
3. Core Subsystems

Subsystem	Purpose
Repository Analyzer	Clones repositories, discovers files and extracts source-code facts.
Dependency Graph	Represents imports, calls, inheritance, DI and other relationships.
Knowledge Tree	Organizes modules, packages, components, classes and responsibilities.
RAG	Retrieves relevant source-code chunks for a question.
AI Agent	Combines RAG, graph traversal, symbol search and code inspection.
Frontend	Displays repository status, tree, graph, source code and chat.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React + TypeScript	Interactive repository UI and chat.
Graph UI	React Flow	Dependency graph visualization.
Backend	Python + FastAPI	REST APIs and orchestration.
AI	LangChain	RAG components, prompts, tools and model integration.
Agent workflow	LangGraph	Stateful multi-step agent workflows.
Code parser	Tree-sitter	Language-aware source-code analysis.
Database	PostgreSQL	Relational application and repository metadata.
Vector search	pgvector	Semantic search inside PostgreSQL.
Cache/jobs	Redis	Caching, progress, rate limiting and job state.
Repository	GitHub API + Git	Metadata and repository cloning.
ORM	SQLAlchemy	Database access.
Validation	Pydantic	Request/data validation.
Testing	Pytest	Backend, parser and RAG tests.
Containers	Docker + Compose	Fully containerized execution.

5. High-Level Architecture



6. Repository Ingestion Pipeline

1. Validate GitHub URL
2. Extract owner and repository
3. Clone repository inside a worker container
4. Detect languages
5. Scan source files
6. Ignore generated, binary and dependency directories
7. Parse source code
8. Extract symbols and relationships
9. Build dependency graph
10. Build knowledge tree
11. Split source code into semantic chunks
12. Generate embeddings
13. Store metadata, graph data and vectors
14. Mark indexing job as COMPLETED

7. Files to Ignore

Directories: .git, node_modules, target, build, dist, venv, __pycache__, .idea, .vscode, coverage and vendor.

Binary/generated examples: .png, .jpg, .jpeg, .gif, .zip, .jar, .exe, .dll, .so and generated build artifacts.

8. Code Analysis with Tree-sitter

Use AST/static analysis for facts that can be extracted exactly from code. Use the LLM for semantic descriptions and explanations.

Language	Initial approach
Java	Tree-sitter Java grammar: classes, methods, imports, inheritance, calls.
Python	Tree-sitter Python grammar: classes, functions, imports, calls.
JavaScript	Tree-sitter JavaScript grammar: modules, functions, classes, imports.
TypeScript	Tree-sitter TypeScript grammar: types, classes, functions, imports.
C++ / Go	Add after the first languages are stable.

9. Symbols to Extract

- Files and directories
- Packages / modules
- Classes and interfaces
- Functions and methods
- Constructors
- Imports and exports
- Inheritance and interface implementation
- Method/function calls
- Dependency injection relationships
- Annotations / decorators where useful

10. Dependency Graph

The graph represents how components are connected. Relationships should be typed rather than storing only a generic edge.

```

PaymentController
■
■ DEPENDS_ON
▼
PaymentService

```

```

■
■ USES
▼
PaymentRepository

Possible edge types:
IMPORTS
CALLS
DEPENDS_ON
EXTENDS
IMPLEMENTS
USES
INJECTS
CONTAINS
PUBLISHES
CONSUMES

```

11. Graph Data Model

```

graph_nodes
-----
id
repository_id
type
name
file_id
symbol_id

graph_edges
-----
id
repository_id
source_node_id
target_node_id
type
metadata

```

For V1, keep graph data in PostgreSQL tables. Introduce Neo4j only if graph traversal later becomes complex enough to justify a dedicated graph database.

12. Knowledge Tree

The dependency graph answers 'what depends on what?'. The knowledge tree answers 'how should a human understand this repository?'.

```

Repository
■
■ ■ ■ Authentication
■ ■ ■ ■ Controllers
■ ■ ■ ■ ■ AuthController
■ ■ ■ ■ ■ Services
■ ■ ■ ■ ■ AuthService
■ ■ ■ ■ ■ Security
■ ■ ■ ■ ■ JwtFilter
■
■ ■ ■ Payments
■ ■ ■ ■ Controllers
■ ■ ■ ■ ■ PaymentController
■ ■ ■ ■ ■ Services
■ ■ ■ ■ ■ PaymentService
■ ■ ■ ■ ■ Repositories
■ ■ ■ ■ ■ PaymentRepository
■
■ ■ ■ Users
■ ■ ■ ■ UserController
■ ■ ■ ■ ■ UserService
■ ■ ■ ■ ■ UserRepository

```

The initial hierarchy comes from the filesystem and code structure. The LLM can enrich nodes with concise descriptions and responsibilities.

13. RAG Pipeline

```

Source File
↓
Semantic / AST-aware Chunk
↓
Embedding Model
↓
pgvector
↓
User Question
↓
Query Embedding
↓
Vector Search
↓
Metadata / Symbol Filtering
↓
Graph Expansion
↓
Reranking
↓
Context Builder
↓
LLM
↓
Answer + File / Line References

```

14. Code Chunk Metadata

```
{
  "repository_id": 123,
  "file": "src/payment/PaymentService.java",
  "class": "PaymentService",
  "method": "processPayment",
  "language": "java",
  "start_line": 42,
  "end_line": 91,
  "commit": "abc123"
}
```

Metadata enables precise source references such as `src/payment/PaymentService.java:42-91`.

15. Hybrid Retrieval

Do not depend on vector search alone. Combine semantic, keyword, symbol and graph retrieval.

Component	Percentage
User Query	1%
Vector Search	33%
Keyword Search	33%
Symbol Search	33%
Reranker	1%
Graph Expansion	1%
Final Context	1%
LLM	1%

16. Example: 'Where is createOrder called?'

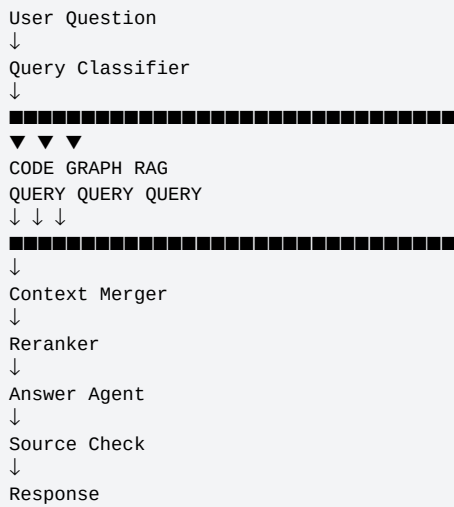
```

User question
↓
Symbol search: createOrder()
↓
Find definition + references
↓
Dependency graph
↓
OrderController
↓
OrderService.createOrder()
↓
Retrieve relevant source chunks

```

↓
LLM explains call path and cites files/lines

17. LangGraph Agent Architecture



18. Agent Tools

- search_code()
- read_file()
- search_symbol()
- find_references()
- find_dependencies()
- find_dependents()
- get_file_structure()
- get_knowledge_tree()
- get_graph_path()
- get_git_history()
- search_documentation()


```
"status": "QUEUED"
}
```

23. Database Schema

Table	Important fields
users	id, identity fields, created_at
repositories	id, github_url, owner, name, default_branch, commit_hash, status
repository_branches	id, repository_id, name, commit_hash
files	id, repository_id, path, language, size, hash
symbols	id, file_id, name, type, start_line, end_line, parent_symbol_id
dependencies	id, source_symbol_id, target_symbol_id, type
code_chunks	id, file_id, symbol_id, content, start_line, end_line, embedding
indexing_jobs	id, repository_id, status, progress, error, timestamps
knowledge_nodes	id, repository_id, parent_id, name, type, description, symbol_id
conversations	id, repository_id, user_id, timestamps
messages	id, conversation_id, role, content, sources, timestamp

24. API Design

```
POST /api/repositories
GET /api/repositories
GET /api/repositories/{id}

POST /api/repositories/{id}/index
GET /api/repositories/{id}/index-status

GET /api/repositories/{id}/tree
GET /api/repositories/{id}/graph

GET /api/repositories/{id}/files
GET /api/repositories/{id}/files/{file_id}

POST /api/repositories/{id}/chat

GET /api/repositories/{id}/symbols/{symbol}
GET /api/repositories/{id}/dependencies/{symbol}
GET /api/repositories/{id}/references/{symbol}
```

25. Frontend Requirements

[illegible]

26. Incremental Indexing

```
Old Commit
↓
New Commit
↓
git diff
↓
Changed Files
↓
Re-parse changed files
↓
Update dependencies
↓
Re-chunk changed symbols
↓
Re-embed changed chunks
```

This avoids re-indexing an entire repository after every small commit.

27. Git History Features

- Why was this class changed?
- Which files frequently change together?
- What changed between two commits?
- Which module has the highest churn?
- What components were affected by a recent change?

28. Source Citations

Authentication is implemented through JwtFilter and AuthService.

Sources:

- src/security/JwtFilter.java:24-67
- src/auth/AuthService.java:31-82
- src/config/SecurityConfig.java:15-43

Source provenance should be preserved from retrieval all the way to the final answer.

29. Recommended Project Structure

```
github-ai-assistant/
├── docker-compose.yml
├── .env.example
├── README.md
├──
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── app/
│   │   ├── main.py
│   │   └── api/
│   │       ├── repositories.py
│   │       ├── indexing.py
│   │       ├── graph.py
│   │       ├── chat.py
│   │       ├── models/
│   │       ├── schemas/
│   │       ├── services/
│   │       ├── github_service.py
│   │       ├── repository_service.py
│   │       ├── parser_service.py
│   │       ├── graph_service.py
│   │       ├── knowledge_service.py
│   │       └── rag_service.py
│   ├── parsers/
│   │   ├── java_parser.py
│   │   ├── python_parser.py
│   │   └── javascript_parser.py
```

```
■ ■■■ agents/
■ ■ ■■■ query_agent.py
■ ■ ■■■ graph_agent.py
■ ■ ■■■ answer_agent.py
■ ■■■ rag/
■ ■ ■■■ embeddings.py
■ ■ ■■■ retriever.py
■ ■ ■■■ reranker.py
■ ■ ■■■ prompts.py
■ ■■■ workers/
■ ■■■ indexing_worker.py
■
■ ■■■ frontend/
■ ■■■ Dockerfile
■ ■■■ package.json
■ ■■■ src/
■
■ ■■■ data/
```

30. Environment Variables

```
LLM_API_KEY=...
DATABASE_URL=postgresql+psycopg://...
REDIS_URL=redis://redis:6379/0
GITHUB_TOKEN=...
```

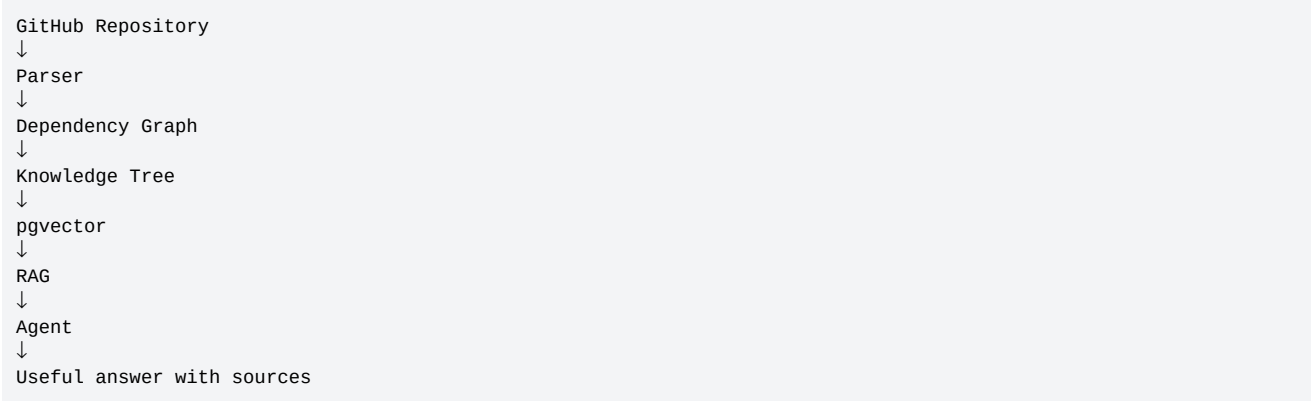
Never hardcode API keys. Use `.env` locally and pass them through Docker Compose. A GitHub token can be optional for public repositories initially, but is useful for higher API limits.

31. Implementation Roadmap

Phase	Focus	Deliverable
1	Docker foundation	Compose, React, FastAPI, PostgreSQL, Redis; one-command startup.
2	GitHub ingestion	Validate URL, clone repository in worker, discover files and track status.
3	Code analysis	Tree-sitter; symbols, imports, inheritance, calls and DI relationships.
4	Graph + tree	Persist dependency graph and generate knowledge tree.
5	RAG	Chunking, embeddings, pgvector, retrieval and source metadata.
6	Hybrid retrieval	Vector + keyword + symbol + graph search and reranking.
7	Agents	LangGraph workflow and repository-analysis tools.
8	Frontend intelligence	Tree, graph, source viewer, indexing progress and chat.
9	Production features	Incremental indexing, caching, rate limiting, observability and evaluation.

32. What Not to Add Initially

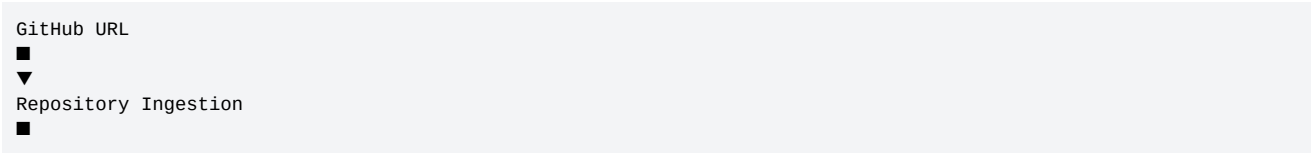
Avoid premature infrastructure: Kubernetes, Kafka, Neo4j, Elasticsearch, multiple LLM providers, many microservices, complex authentication and autonomous production actions.

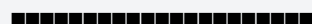


33. AI Coding Agent Build Strategy

- Milestone 1: Create Docker Compose and verify base containers start.
- Milestone 2: Implement GitHub URL validation and repository cloning.
- Milestone 3: Implement file filtering and Tree-sitter parsing.
- Milestone 4: Implement symbols, dependencies and graph persistence.
- Milestone 5: Implement knowledge tree generation.
- Milestone 6: Implement code chunking, embeddings and pgvector retrieval.
- Milestone 7: Implement hybrid retrieval and source citations.
- Milestone 8: Implement LangGraph agents and tools.
- Milestone 9: Implement React graph/tree/chat UI.
- Milestone 10: Add incremental indexing, caching, tests and observability.

34. Final Recommended Architecture





35. Host Environment Requirement



- ```

most
■■■ Python
■■■ Docker Desktop / Docker Engine

```

## Docker

- ```

■■■ frontend
■■■ backend
■■■ worker
■■■ postgres + pgvector
■■■ redis

```

The intended model is Docker-first. The host does not need application runtimes or databases installed separately.

36. Success Criteria

- A user can paste a public GitHub URL and start indexing.
- Indexing runs asynchronously and exposes progress.
- The system displays a repository knowledge tree.
- The system displays dependency relationships.
- The system supports semantic and symbol search.
- The assistant answers repository questions using retrieved context.
- Answers cite relevant files and line ranges.

- Changed commits can be incrementally re-indexed.
- The complete stack starts with Docker Compose.
- No database, cache, Git, Node.js or AI framework needs to be installed on the host.

Recommended V1 principle: Keep the first release small and reliable. Start with one or two programming languages, PostgreSQL + pgvector, Redis, Tree-sitter, LangChain/LangGraph and Docker Compose. Add more infrastructure only after the end-to-end RAG workflow is working.